

Security Assessment and Hardening of a Student Registration Web Application

IFT 542 — Web Application Security · Practical Assignment

Name: Abayomi Favour T.
Matriculation number: 2021/01/84091CF
Date: 03 August 2026
Implementation: Python 3.12 / Flask 3.1 / SQLite 3 / Argon2id

Authorised-lab statement. Every test described in this report was performed against an isolated instance of the application bound to the loopback interface (127.0.0.1) of a machine under my sole control. No system belonging to FUT Minna, no public website and no third-party service was scanned, probed or contacted at any point. All accounts, documents and data values are fictitious. Reusable attack tooling is not included in this submission; weaknesses are demonstrated only through automated tests inside the isolated application, as the brief requires.

1. Executive summary

The prototype was reviewed against the STRIDE model, and 11 threats were identified across all six categories. Two defects dominated the risk profile: the login statement was assembled by pasting submitted values into SQL text, and passwords were stored in a directly comparable form so that they could be tested inside that statement. Together these allowed the credential check to be removed from the query altogether, and ensured that any single read of the user table would disclose every account's password in clear text.

The application was then rebuilt with the defects corrected and defences added for the four vulnerability classes required by Task 3. All 112 automated tests pass. The three highest risks were reduced from scores of 25, 20 and 16 to residual scores of 5, 4 and 4 respectively; three lower-ranked risks are formally accepted at their residual level, with justification in §2.5. Evidence items E1-E13 in [evidence/captures/](#) are live output captured from the running application, not illustrations.

Headline outcome. The corrected data-access layer makes statement structure independent of user input; a static check that parses every module's syntax tree confirms no SQL anywhere in the application is built from a value, and that check runs inside the test suite so a regression fails the build.

2. Task 1 — STRIDE threat model and risk assessment

2.1 System decomposition and data-flow diagram

The application supports student login, profile update, course registration, document upload, an administrative area, and a URL-preview feature that imports a document reference on the user's behalf. The diagram below decomposes it into external entities, processes, data stores and the flows between them, with five trust boundaries marked. A boundary is drawn wherever data crosses from one level of trust to another; those crossings are where the threats in §2.2 are concentrated.

Student Registration Web Application — Level 1 Data-Flow Diagram

IFT 542 Practical Assignment · dashed red lines are trust boundaries

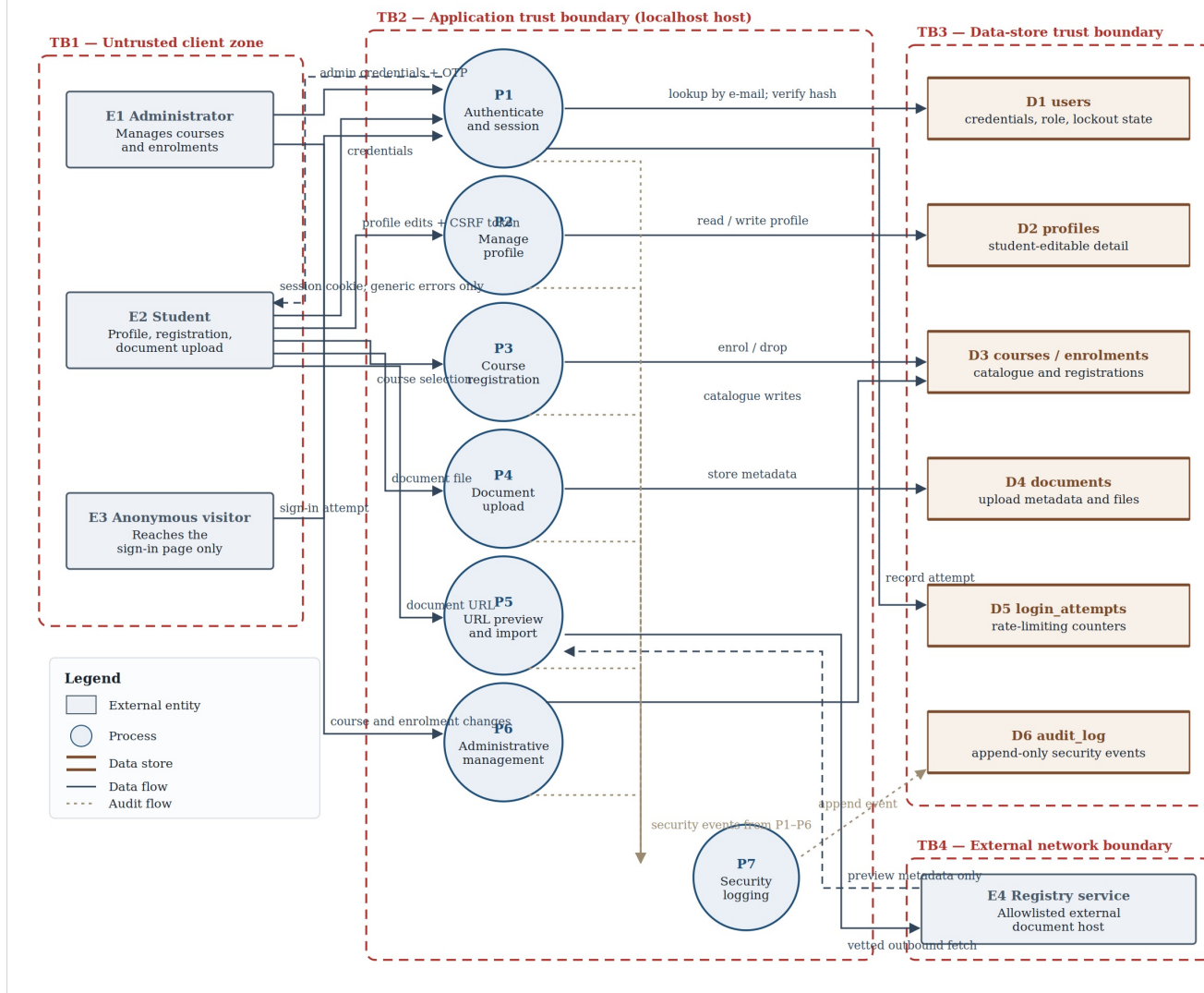


Figure 1 — Level 1 data-flow diagram with trust boundaries. Source: [docs/dfd.png](#), generated by `scripts/make_dfd.py`.

The boundaries carry different assumptions. **TB1** separates the browser, which is entirely attacker-controllable, from everything else: nothing arriving across it may be trusted, including hidden fields, headers and cookies. **TB2** encloses the application processes. **TB3** protects the data stores, of which D1 is the most sensitive because it holds credential material. **TB4** governs outbound traffic; it exists because P5 makes the server issue requests on a user's behalf, which is what makes server-side request forgery possible at all.

2.2 STRIDE worksheet

11 application-specific threats were identified, covering every STRIDE category. These are threats to this system rather than generic categories: each names the element it acts on and the consequence it produces.

STRIDE category	Threats	IDs
Spoofing	1	T-04
Tampering	2	T-01, T-08
Repudiation	1	T-09
Information Disclosure	3	T-02, T-06, T-07
Denial of Service	2	T-10, T-11
Elevation of Privilege	2	T-03, T-05

Table 1 — STRIDE coverage. Full worksheet: [docs/stride_worksheet.csv](#).

ID	Category	Application-specific threat	DFD elements
T-01	Tampering	Authentication bypass through SQL injection in the login form	P1, D1, D3
T-02	Information Disclosure	Mass credential disclosure from plaintext password storage	D1
T-03	Elevation of Privilege	Administrator session takeover via stored cross-site scripting	P2, P6, D2
T-04	Spoofing	Impersonation of a student through credential stuffing	E2, P1, D1
T-05	Elevation of Privilege	Forced browsing to administrative functions	P6, D1, D3
T-06	Information Disclosure	Internal network probing through the URL-preview feature	P5, E4
T-07	Information Disclosure	Schema and account enumeration through verbose responses	P1
T-08	Tampering	Forged course registration via cross-site request forgery	E2, P2, P3, D3

T-09	Repudiation	Denial of a registration or enrolment change	P3, P6, D6
T-10	Denial of Service	Resource exhaustion through unbounded document upload	P4, D4
T-11	Denial of Service	Deliberate lockout of legitimate student accounts	P1, D1

Table 2 — STRIDE worksheet (abridged; the CSV carries the full description of each threat).

2.3 Risk register

Each threat was scored for likelihood and impact on a 1–5 scale, with **Risk = Likelihood × Impact**. Likelihood reflects how readily the threat could be realised against this application as found; impact reflects the consequence for the institution and for the students whose data it holds. Bands are Critical (20–25), High (12–19), Moderate (6–11) and Low (1–5).

#	ID	Threat	L	I	Risk	Band	Residual	Band
1	T-01	Authentication bypass through SQL injection in the login form	5	5	25	Critical	1×5=5	Low
2	T-02	Mass credential disclosure from plaintext password storage	4	5	20	Critical	1×4=4	Low
3	T-03	Administrator session takeover via stored cross-site scripting	4	4	16	High	1×4=4	Low
4	T-04	Impersonation of a student through credential stuffing	5	3	15	High	2×3=6	Low
5	T-05	Forced browsing to administrative functions	3	5	15	High	1×5=5	Low
6	T-06	Internal network probing through the URL-preview feature	3	4	12	High	1×4=4	Low
7	T-07	Schema and account enumeration through verbose responses	4	3	12	High	1×3=3	Low
8	T-08	Forged course registration via cross-site request forgery	3	3	9	Moderate	1×3=3	Low
9	T-09	Denial of a registration or enrolment change	3	3	9	Moderate	2×3=6	Low
10	T-10	Resource exhaustion through unbounded document upload	3	3	9	Moderate	2×2=4	Low
11	T-11	Deliberate lockout of legitimate student accounts	3	2	6	Low	3×2=6	Low

Table 3 — Risk register with residual risk after controls. Source: [docs/risk_register.csv](#).

2.4 The three highest-priority risks

T-01 — Authentication bypass through SQL injection in the login form (5×5 = 25, Critical)

Ranked first because it scores maximum on both axes. Likelihood is 5 because the flaw sits on an unauthenticated endpoint reachable by anyone who can load the login page, and requires no special tooling to trigger. Impact is 5 because success does not merely leak data: the injection point is the authentication check itself, so the attacker becomes an authenticated user, and the same pattern in the course search grants read and write access across every table. This is the one finding that compromises confidentiality, integrity and availability simultaneously. **Control:** Replace every statement with a parameterised query so values bind as data after parsing; retrieve the account by identifier only; validate e-mail type, length and format before the data layer; route all access through a single db.query() helper. **Residual 5** — likelihood falls to 1 because statement structure is now fixed before any value is attached; impact is left at 5 because the consequence of a future regression would be just as severe, which is precisely why the static check runs in the test suite.

T-02 — Mass credential disclosure from plaintext password storage (4×5 = 20, Critical)

Ranked second because it multiplies the damage of every other confidentiality failure. Likelihood is 4 rather than 5 because it needs a prior read of the table — but T-01 supplies exactly that, and so would a mislaid backup or a curious insider. Impact is 5 because the loss is not recoverable by patching: once disclosed, the passwords are disclosed, and since students reuse passwords the harm propagates to email and banking accounts the university does not control. **Control:** Store Argon2id hashes (m=19456, t=2, p=1) with a unique per-password salt; verify with the library's constant-time function; re-hash transparently when parameters change; never place a password in a SQL statement. **Residual 4** — an attacker who now obtains the table holds only memory-hard hashes with unique salts, which must be attacked one password at a time.

T-03 — Administrator session takeover via stored cross-site scripting (4×4 = 16, High)

Ranked third because it converts an ordinary student account into an administrative one. Likelihood is 4 because storing the payload requires nothing more than editing one's own profile, and the administrator will open that record in the normal course of duty. Impact is 4 because the script runs with the administrator's session and can alter courses and enrolments; it is not 5 because the credential store itself is not directly exposed. **Control:** Render every user value through the auto-escaping template engine with no escape bypass; add a nonce-based Content-Security-Policy with no 'unsafe-inline'; set HttpOnly on the session cookie so a script cannot read it even if one executes. **Residual 4** — three independent layers must fail together: escaping, the nonce-based policy, and the HttpOnly flag that keeps the cookie unreadable even if a script did execute.

2.5 Residual risk and accepted risks

Three risks remain accepted rather than fully mitigated, because the residual cause is inherent to the control rather than a gap in it.

- **T-04 (residual 6)** — lockout and rate limiting slow credential stuffing but cannot stop an attacker who already holds a correct password. Accepted because the remaining exposure is addressed by detection (every attempt is logged) rather than prevention; extending MFA to all students would reduce it further at a usability cost the department has not agreed to.
- **T-09 (residual 6)** — the audit log demonstrates what happened, but it is written to the same host as the application, so an attacker with root could alter it. Accepted for the lab; off-host forwarding to write-once storage is the recommended next control.
- **T-11 (residual 6, unchanged)** — deliberate lockout of a legitimate student is *caused* by the control introduced for T-04. The score is deliberately not reduced, because pretending otherwise would hide a real trade-off. It is bounded instead: the lock expires after 15

minutes rather than persisting, it clears on the next successful sign-in, and lockout events are logged so a targeted pattern is visible. Accepted on the basis that a 15-minute delay to one student is a smaller harm than an unthrottled path to account compromise.

3. Task 2 — Secure authentication and SQL injection remediation

3.1 The unsafe pattern and the affected data flow

The defective statement is on the flow E2/E1 → P1 → D1 in Figure 1 — the crossing of TB1, where wholly untrusted input reaches the credential store. The prototype built the statement by interpolating both submitted values into the SQL text:

```
insecure_baseline/legacy_login.py :: legacy_authenticate() (BEFORE)

def legacy_authenticate(conn, email: str, password: str):
    sql = (
        "SELECT id, email, role FROM users "
        f"WHERE email = '{email}' AND password = '{password}'"
    )
    cursor = conn.execute(sql)          # structure now depends on input
    return cursor.fetchone(), sql
```

Two distinct defects are present. First, the values are pasted into the statement *before* the engine parses it, so the engine cannot distinguish characters the developer intended as command from characters the user supplied as data. A quote inside the submitted value ends the string literal early, and everything after it is parsed as SQL. The submitted data therefore changes what the statement *means*, not merely what it matches — and because the statement is the authentication check, its meaning can be changed to one that no longer tests the password at all.

Second, comparing the password inside SQL forces the stored value to be directly comparable, which requires plaintext or an unsalted digest. The two defects are linked: fixing the query without fixing the storage would leave the credential exposure untouched.

3.2 The corrected pattern

```
secure_app/db.py :: find_user_by_email() (AFTER)

def find_user_by_email(email: str):
    """Retrieve the account by identifier ONLY. The password is not part
    of the query; it is verified afterwards against the stored hash."""
    return query(
        "SELECT id, email, matric_no, full_name, password_hash, role, "
        "      mfa_secret, mfa_enabled, failed_attempts, locked_until "
        "FROM users WHERE email = ?",
        (email,),
        one=True,
    )
```

```
secure_app/auth.py :: login() (AFTER)

# secure_app/auth.py :: login()
user = db.find_user_by_email(email)          # bound parameter, data only
stored = user["password_hash"] if user else None
if not security.verify_password(stored, password): # Argon2id, in app code
    ...
    flash(GENERIC_LOGIN_ERROR, "error")      # one message for every failure
    return render_template("login.html"), 401
```

Three things changed. The statement text is now fixed and contains a placeholder; the value travels separately as a bound parameter; and the password has left the query entirely, so the account is retrieved by identifier and the credential is verified afterwards in application code.

3.3 How parameterisation separates data from code

The mechanism. With a parameterised statement the database receives the SQL text and the values through separate channels. The engine parses the text first and produces an execution plan while the placeholder is still an empty slot. Only then are the supplied values bound into those slots. Because parsing has already finished, no character inside a value can be promoted to syntax — there is no parser still running for it to influence. A submitted quote is simply a quote character in a string.

This is why parameterisation is a genuine fix and escaping is not. Escaping tries to neutralise dangerous characters while still concatenating them into code, so it depends on the developer correctly anticipating every character the engine treats as special, in every context and character set. Parameterisation removes the attacker's influence over structure altogether, which is the property that made the flaw exploitable. Input validation is retained as a second layer, but it is not the control that closes the vulnerability.

The comparison was run against both implementations with identical input. The legacy statement matched an account without a correct credential; the parameterised statement returned nothing, because it searched for an account whose email address is literally that string. Both still authenticate a genuine account correctly.

Evidence: **E7_sqli_before_after.txt** (side-by-side run), **E10_static_sql_scan.txt** (static check across every module).

3.4 Password storage

```
secure_app/security.py :: password hashing
```

```
_hasher = PasswordHasher(
    time_cost=2,          # iterations
    memory_cost=19456,    # 19 MiB - memory-hard, resists GPU/ASIC cracking
    parallelism=1,
    hash_len=32, salt_len=16,
    type=Type.ID,         # Argon2id
)

# Verifying against a dummy hash when the account does not exist keeps the
# response time of "unknown user" close to "known user, wrong password".
_DUMMY_HASH = _hasher.hash("not-a-real-password-" + secrets.token_hex(8))
```

Argon2id is used at the OWASP baseline of 19 MiB memory, two iterations and one degree of parallelism. It is memory-hard, so an attacker cannot trade cheap parallel hardware for speed as they can against iteration-only functions. The library generates a unique salt per password and embeds it in the encoded hash, so identical passwords never produce identical stored values and precomputed tables do not apply. Verification uses the library's constant-time comparison, and a stored hash written under weaker parameters is transparently upgraded on the next successful login.

Inspection of the seeded database confirms every stored value carries the `$argon2id$v=19$m=19456,t=2,p=1$` prefix with a distinct salt, that no known lab password appears anywhere in the stored values, and that no table in the schema declares a plaintext password column.

Evidence: [E6_password_storage.txt](#).

3.5 Supplementary authentication controls

The brief requires at least two additional controls; four were implemented.

- **Temporary account lockout** — five failures lock the account for 15 minutes. Verified live: the counter reaches the threshold, the lock is recorded with an expiry, and the correct password is then refused *using the identical wording as an ordinary failure*, so the response does not tell an attacker that the account exists and is locked.
- **Per-source rate limiting** — 20 failures from one address in a 15-minute window returns HTTP 429. This targets password spraying, which lockout alone does not address because it spreads few attempts across many accounts.
- **TOTP multi-factor for the privileged role** — RFC 6238, over the standard library. The administrator's password alone establishes no session: it redirects to the second-factor challenge, and the session is created only after a valid code.
- **Session identifier regeneration** — the session is cleared and a new identifier and CSRF token minted at every privilege change, defeating session fixation and preventing a pre-authentication token from carrying over.

Two supporting measures address enumeration: every authentication failure returns one generic message, and when the account does not exist the code still performs a full verification against a dummy hash so that response timing does not disclose account existence.

Evidence: [E2_authentication_outcomes.txt](#), [E8_lockout_and_ratelimit.txt](#).

3.6 Test results

All 112 tests pass. The suite proves valid login succeeds, invalid credentials are rejected, unsafe input does not change query meaning, and stored passwords are not plaintext, alongside the supplementary controls. Two checks are worth singling out because they guard against tests that appear to pass while proving nothing: the static SQL scanner is itself run against a file containing all four unsafe patterns to confirm it fires, and against the legacy baseline, which it flags at the two known defective lines.

Evidence: [E9_test_results.txt](#).

4. Task 3 — Web defences, incident response and ethics

4.1 Cross-site scripting

The protected field is the profile biography, rendered on the dashboard. The value is *not* sanitised on input: it is stored exactly as typed and made safe at the point of output. Encoding at output is context-aware and remains correct if the value is later reused in a different context; input filtering is neither, and silently corrupts legitimate data such as an ampersand in a name.

Rendering goes through the auto-escaping template engine, with no escape bypass anywhere in the project — a test asserts that no template uses the `|safe` filter and no module constructs `Markup()`. Behind that sits a nonce-based Content Security Policy: `script-src 'self' 'nonce-<random>'` with no `'unsafe-inline'`, so injected script lacking the per-response nonce is not executed. The nonce is regenerated per response.

Verified live: a stored marker containing markup metacharacters is returned in the response as HTML entities, while the database confirms the original value was stored unmodified.

Evidence: [E5_xss_encoding.txt](#), [E1_http_security_headers.txt](#).

4.2 Cross-site request forgery

Course registration and profile update are protected with the synchroniser token pattern. A random token is bound to the session and must be echoed on every state-changing request. An attacker's page can cause the browser to send a cross-site POST, but the same-origin policy prevents it from reading the victim's token, so the forged request fails validation. Enforcement is central rather than per-view:

```
secure_app/_init_.py :: before_request (CSRF enforcement)

@app.before_request
def _before():
    ...
    # Enforced centrally for every unsafe method. A view cannot opt out by
    # omission; it would have to be added to an explicit exemption set.
    if request.method in {"POST", "PUT", "PATCH", "DELETE"}:
        submitted = (request.form.get(security.CSRF_FIELD)
                     or request.headers.get("X-CSRF-Token"))
        if not security.csrf_is_valid(g.csrf_token, submitted):
            security.audit("csrf.rejected", "denied", ...)
            abort(403)
```

Tokens are compared in constant time. `SameSite=Lax` on the session cookie is an independent second layer: the browser withholds the cookie from cross-site POSTs entirely, so the request arrives unauthenticated even before token validation. Verified live: a POST with no token and a POST with a forged token both return 403, the token issued to one session does not authorise a request in another, and the correct token succeeds.

Evidence: [E3_csrf_enforcement.txt](#).

4.3 Server-side request forgery

The URL-preview feature makes the server issue a request on the user's behalf, which without restriction turns it into a proxy into the trusted network. Six controls are applied in order: scheme allowlist (http/https only), port allowlist, host allowlist, DNS resolution with every returned address checked, address pinning, and redirects disabled.

```
secure_app/security.py :: validate_outbound_url()

# (c) host allowlist, checked BEFORE resolution so a blocked host
#     causes no network request at all
if host not in allow:
    raise SSRFBlocked("Destination host is not on the approved list.")

# (d) resolve and vet EVERY returned address
for addr in addresses:
    if not address_is_public(ipaddress.ip_address(addr)):
        # One private answer condemns the whole name: a host resolving to
        # both a public and an internal address is a rebinding attempt.
        raise SSRFBlocked("Destination resolves to a restricted network range.")

# (e) pin the vetted address for the actual connection
vetted_ip = addresses[0]
```

Two details matter more than the allowlist itself. Checking *every* resolved address and rejecting the name if any one is internal defeats DNS rebinding, where an approved name answers with an internal address. Pinning the connection to the address that was vetted closes the time-of-check to time-of-use gap between validation and connection. Disabling redirects matters because a 302 response would otherwise be followed to a destination that was never checked. The host allowlist is evaluated before resolution, so a blocked host generates no DNS lookup at all — confirmed by a test that counts resolver calls.

Verified live against loopback, private, metadata, non-HTTP-scheme, non-standard-port, unlisted-host and embedded-credential destinations: all refused, each logged. The refusal wording is deliberately uniform so the endpoint cannot be used as an internal-network oracle.

Evidence: [E4_ssrf_policy.txt](#).

4.4 Security misconfiguration

- **No default credentials.** The seed script generates passwords at runtime and prints them once; nothing is hardcoded. A test scans the source for credential-like assignments.

- **Debug disabled** by default, and the production profile *refuses to start* without a SECRET_KEY rather than falling back to a guessable default — asserted by a test.
- **Secrets kept out of the repository** via `.env` (git-ignored) with a committed template.
- **Security headers** applied centrally: CSP, X-Content-Type-Options, X-Frame-Options: DENY, Referrer-Policy, Permissions-Policy, COOP/CORP and Cache-Control: no-store. HSTS is emitted only over TLS, since it is meaningless on a plain-HTTP lab origin. The server banner is overwritten so the framework and version are not disclosed.
- **Generic error handling**. Unhandled exceptions return a reference code; the detail goes only to the server log. This closes the verbose-error defect that let the prototype disclose schema information.
- **Upload hardening**. Extension and declared-type allowlists, a magic-byte check that the content matches the claimed type, a 2 MiB ceiling, and a server-generated random storage name outside any served directory with 0600 permissions — so a supplied name can neither traverse directories nor set an executable extension.
- **Dependencies** pinned in `requirements.txt`: Flask 3.1.3, argon2-cffi 25.1.0, requests 2.33.1, pytest 8.4.2, with no known advisories outstanding at the time of writing.

Evidence: [E1_http_security_headers.txt](#), [E12_access_control.txt](#), [E13_upload_validation.txt](#).

4.5 Security logging

Failed logins, denied authorisations and rejected validation are recorded as structured JSON events answering who, what, when and outcome.

```
secure_app/security.py :: redact()

SENSITIVE_KEYS = {"password", "csrf_token", "session", "cookie",
                  "otp", "mfa_code", "totp", "mfa_secret", "secret", ...}

def redact(payload: dict) -> dict:
    """Strip secret-bearing keys before anything is written to a log."""
    clean = {}
    for key, value in payload.items():
        if key.lower() in SENSITIVE_KEYS:
            clean[key] = "[REDACTED]"
        elif key.lower() in {"email", "username"}:
            clean[key] = mask_email(str(value)) # g***y@example.test
        else:
            clean[key] = value
    return clean
```

Secrets are stripped by key before anything is written, and email addresses are masked so the log retains the domain for triage without recording the full personal identifier. Actors are internal user IDs rather than names, which keeps the personal data in the log to the minimum the record needs to be useful. An automated check greps the produced log for every lab password, the full email address and the session cookie: none appear.

Evidence: [E11_security_log.txt](#) (40 events across 14 event types), [evidence/logs/security.log](#).

4.6 Incident response

A one-page runbook covers all six stages — Preparation, Identification, Containment, Eradication, Recovery and Lessons Learned — with named log sources, alert thresholds, a P1–P3 severity scale, and the instruction to preserve evidence with recorded hashes before changing anything. Containment distinguishes short-term actions from longer-term ones and prefers disabling a single feature by configuration over taking the whole service down. Eradication requires a regression test that fails against the vulnerable version, so a fix is not accepted on assertion alone.

The runbook was exercised against a genuine event: the lockout sequence generated during evidence capture was worked through as incident **INC-2026-0003** and closed as P3, attempted account compromise blocked by control.

Documents: [docs/runbook.md](#), [docs/incident_record.md](#).

4.7 Ethics and lawful conduct

A signed declaration accompanies this submission at [docs/ethics_declaration.md](#). It records that all testing was confined to a locally-run instance on the loopback interface, that no university, public or third-party system was contacted, that no real person's credentials or personal data were used, and that the internal addresses named in the SSRF evidence appear only as inputs the application *refused*. The `.test` domain used throughout is reserved by RFC 6761 for exactly this purpose and resolves to nothing.

5. A defect found during evidence capture

Capturing the response headers with `curl -I` returned HTTP 401 from the login page, which should have been 200. The cause was that the views tested `request.method == "GET"`. Flask allows HEAD implicitly and routes it to the GET handler, but `request.method` remains "HEAD", so a HEAD request fell past that branch into the credential-processing path, failed validation and emitted a spurious failed-login audit event.


```
# Before (defective): Flask allows HEAD implicitly and maps it to the GET
# handler, but request.method stays "HEAD", so a HEAD request fell through
# into the credential-processing branch.
if request.method == "GET":
    return render_template("login.html")

# After:
if request.method != "POST":
    return render_template("login.html")
```

The security consequence is not the wrong status code. Any monitoring probe, link checker or search crawler issuing HEAD would have written false failed-login events into the audit trail, degrading the evidence that Task 3 depends on, and would have driven accounts toward lockout — turning a detection control into a denial-of-service vector. It is recorded here because it was found by capturing evidence rather than by reading the code, which is the practical argument for producing evidence from a running system rather than describing intended behaviour. Two regression tests now cover it.

6. Conclusion

The assessment identified 11 threats across all six STRIDE categories and reduced the three highest from Critical and High to Low residual scores, with three lower risks formally accepted and justified. The two defects that dominated the profile were addressed at their root: statement structure is now independent of user input, and credentials are stored as memory-hard salted hashes that are never placed in a query. Defences for the four required vulnerability classes are in place and evidenced from live output, and logging, incident response and ethical conduct are documented.

Three limitations are stated plainly. Sessions are signed client-side cookies rather than a server-side store, so regeneration means discarding pre-auth state and minting a new identifier; a server-side store would additionally permit server-initiated revocation. Rate-limit counters live in the application database, which suits a single process but would need a shared store behind multiple workers. The audit log is written to the same host as the application, so off-host forwarding to write-once storage should precede any deployment beyond the lab.

7. References

1. OWASP Foundation, *OWASP Top 10:2021 — The Ten Most Critical Web Application Security Risks*, 2021. Used for the classification of A01 Broken Access Control, A03 Injection, A05 Security Misconfiguration and A10 Server-Side Request Forgery.
2. OWASP Foundation, *Password Storage Cheat Sheet*. Source of the Argon2id parameter baseline (19 MiB memory, 2 iterations, 1 degree of parallelism) applied in §3.4.
3. OWASP Foundation, *Cross-Site Request Forgery Prevention Cheat Sheet* and *Server-Side Request Forgery Prevention Cheat Sheet*. Basis for the synchroniser-token pattern in §4.2 and the destination-policy ordering in §4.3.
4. Microsoft, *The STRIDE Threat Model* (Microsoft Learn / Security Development Lifecycle). Methodology used for the decomposition in §2.
5. P. Cichonski, T. Millar, T. Grance and K. Scarfone, *Computer Security Incident Handling Guide*, NIST Special Publication 800-61 Revision 2, 2012. Structure of the incident-response runbook in §4.6.
6. A. Biryukov, D. Dinu and D. Khovratovich, *Argon2: the memory-hard function for password hashing and other applications*, Password Hashing Competition, 2015.
7. M. M'Raihi, S. Machani, M. Pei and J. Rydell, *TOTP: Time-Based One-Time Password Algorithm*, RFC 6238, IETF, 2011. Implemented in §3.5.
8. MITRE, *Common Weakness Enumeration*: CWE-89 (SQL Injection), CWE-79 (Cross-site Scripting), CWE-352 (CSRF), CWE-918 (SSRF), CWE-256/257 (password storage), CWE-209 (information exposure through an error message), CWE-384 (session fixation).
9. M. Stapleton (ed.), *Reserved Top Level Domain Names*, RFC 6761, IETF, 2013. Basis for using the `.test` domain for lab destinations.

Declaration of assistance: the work, design decisions and analysis in this submission are my own. Where reference material listed above informed a control, it is cited at the point of use.

Appendix A — Controls applied and verification

ID	Control type	Control applied	Verified by
T-01	Preventive	Replace every statement with a parameterised query so values bind as data after parsing; retrieve the account by identifier only; validate e-mail type, length and format before the data layer; route all access through a single db.query() helper.	tests/test_sqli.py (11 tests)
T-02	Preventive	Store Argon2id hashes (m=19456, t=2, p=1) with a unique per-password salt; verify with the library's constant-time function; re-hash transparently when parameters change; never place a password in a SQL statement.	tests/test_auth.py, evidence/captures/E6_password_storage.txt
T-03	Preventive	Render every user value through the auto-escaping template engine with no escape bypass; add a nonce-based Content-Security-Policy with no 'unsafe-inline'; set HttpOnly on the session cookie so a script cannot read it even if one executes.	tests/test_defences.py, evidence/captures/E5_xss_encoding.txt
T-04	Detective and Corrective	Temporary account lockout after 5 failures for 15 minutes; per-source-IP limiting of 20 failures in a 15-minute window; deliberately slow Argon2id verification; all attempts recorded to login_attempts and the audit log.	tests/test_auth.py, evidence/captures/E8_lockout_and_ratelimit.txt
T-05	Preventive and Detective	Decorate every privileged route with a server-side role check that reads the role from the session record only; deny by default; log each denial as an authz.denied event; require a second factor for the administrator role.	tests/test_auth.py, evidence/captures/E12_access_control.txt
T-06	Preventive	Scheme and port allowlists; explicit host allowlist checked before resolution; rejection of any name resolving to a loopback, private, link-local, reserved or metadata address; connection pinned to the vetted address to close DNS rebinding; redirects disabled; refusal reason withheld from the client.	tests/test_defences.py, evidence/captures/E4_ssrf_policy.txt
T-07	Preventive	One generic failure message for every authentication outcome; a dummy hash verification when the account is absent so response time does not differ; custom error handlers that return a reference code while the detail goes only to the server log.	tests/test_auth.py, tests/test_defences.py
T-08	Preventive	Per-session synchroniser token validated centrally on every unsafe method and compared in constant time; SameSite=Lax on the session cookie as an independent second layer; token rotated when the session is regenerated.	tests/test_defences.py, evidence/captures/E3_csrf_enforcement.txt
T-09	Detective	Structured audit events recording actor, action, object, outcome, source address and timestamp for every state change and every denial, written to an append-only table and a JSON log; the application holds no UPDATE or DELETE against that table.	tests/test_defences.py, evidence/logs/security.log
T-10	Preventive	2 MiB request ceiling enforced by the framework; extension and declared-type allowlists; magic-byte check that the content matches the claimed type; server-generated random storage name outside any served directory; 0600 permissions.	tests/test_defences.py, evidence/captures/E13_upload_validation.txt
T-11	Corrective	Lockout is time-bounded at 15 minutes rather than permanent and clears on the next successful sign-in; rate limiting is scoped to the source address so one host cannot lock many accounts; lockout events are logged so a targeted pattern is visible.	tests/test_auth.py::test_lockout_is_temporary_not_permanent

Appendix B — Evidence index

Item	Contents	Supports
E1	HTTP security response headers, live capture	Task 3 §4.1, §4.4
E2	Authentication outcomes: valid, wrong password, unknown account, admin MFA redirect, HEAD, anonymous, forbidden	Task 2 §3.5
E3	CSRF: missing, forged and valid token; cookie flags	Task 3 §4.2
E4	SSRF: eight refused destination classes	Task 3 §4.3
E5	XSS: stored value vs. rendered output	Task 3 §4.1
E6	Password storage: Argon2id parameters, truncated hashes, schema check	Task 2 §3.4
E7	SQL injection before/after side-by-side run	Task 2 §3.1–3.3
E8	Account lockout and source rate limiting	Task 2 §3.5
E9	Full pytest transcript (112 tests)	All tasks
E10	Static AST scan for dynamically built SQL, with control file	Task 2 §3.3
E11	Security audit log, redacted, with secret-absence checks	Task 3 §4.5
E12	Access control: vertical, horizontal, forged role	Task 3 §4.4
E13	Upload validation: extension, content, path component	Task 3 §4.4

Appendix C — Reproducing this assessment

<pre>pip install -r requirements.txt python3 scripts/seed.py --fixed python3 run.py python3 -m pytest tests/ -v bash scripts/capture_evidence.sh python3 scripts/threat_model.py python3 scripts/make_dfd.py python3 scripts/build_report.py</pre>	<pre># fictitious accounts # binds 127.0.0.1 only # 112 tests # regenerates E1-E13 # STRIDE worksheet + risk register # data-flow diagram # this report</pre>
--	---

Setup instructions, test accounts and known limitations are in [README.md](#).